

AI Test Case Generator

Local OpenRouter utility with structured validation and one-sheet Excel export

Candidate	Jeremiah Igbinedion
Implementation	TypeScript, Express, Zod, ExcelJS and OpenRouter
Interface	Local browser application at http://127.0.0.1:4173
Output	One validated Test Cases worksheet (.xlsx)
Status	Implemented and verified with mocked and live OpenRouter responses

Purpose

The utility converts pasted feature requirements into a small, traceable set of QA test cases. It calls an LLM through OpenRouter, requires a strict JSON-schema response, validates every case, and creates an Excel workbook only when the complete result passes the quality gate.

1. Design and Data Flow

- The user pastes text into a local light-themed browser interface; no document upload is used.
- The Express server resolves `OPENROUTER_API_KEY` and `OPENROUTER_MODEL` from `.env` or one-request overrides.
- The frontend receives configuration booleans only. Environment values are never returned.
- OpenRouter receives the exact system instruction and an application message containing the specification and generation settings.
- The client requires `response_format` type `json_schema` and provider `require_parameters`.
- Zod and deterministic quality checks validate the result before ExcelJS creates a workbook in memory.
- Browser output downloads directly. CLI output is written only to the ignored `ai/reports` directory.

Security decisions

Runtime credentials apply to one request, are masked in the browser, and are not stored in browser storage, cookies, logs, workbooks, or later sessions. The server binds to localhost and rejects oversized JSON requests. Error responses redact unexpected provider and implementation details.

Output contract

The structured response contains `testCases` and `clarificationQuestions`. Clarification questions support rejection feedback but are not exported. The workbook contains one Test Cases sheet with the required case fields and a human-review disclaimer.

2. Exact System Prompt

You are a senior QA engineer specialising in requirement analysis and risk based test design.

Your responsibility is to generate the smallest complete set of meaningful test cases supported by the supplied specification.

Accuracy, traceability, and business relevance are more important than producing a large number of test cases.

Never invent requirements or expected behaviour.

Never convert an ambiguity into an assumed requirement.

Never generate duplicate cases merely to increase the test case count.

Every test case must map to a supplied requirement and contain an observable expected result.

If the specification does not provide enough information to determine an expected result, create a clarification question instead of guessing.

Before returning the result, audit every case for traceability, duplication, unsupported assumptions, executability, and meaningful coverage.

Return structured JSON that matches the application's output schema.

Prompt design rationale

The prompt prioritises the smallest complete set over test-count inflation. It prohibits invented requirements, forces traceability and observable expected results, and sends uncertainty to clarification questions instead of allowing guessed behaviour. The application adds only the feature name, selected test level, case limit, specification, and output-field contract.

3. Example Input and Structured Output

Example feature specification

R1. Tier 1 users can transfer a maximum of 20,000 per transaction.
R2. Transfers above 20,000 are rejected and no money leaves the wallet.

Example validated JSON

```
{
  "testCases": [{
    "testCaseId": "TC-001",
    "title": "Accept a transfer at the Tier 1 limit",
    "requirementId": "R1",
    "priority": "P1",
    "testType": "Functional boundary",
    "preconditions": ["A Tier 1 wallet has sufficient balance"],
    "testData": ["Transfer amount: 20,000"],
    "steps": ["Submit a transfer for 20,000"],
    "expectedResult": "The transfer of 20,000 is accepted.",
    "traceabilityEvidence": "maximum of 20,000 per transaction",
    "assumptions": "None",
    "reviewStatus": "Ready"
  }],
  "clarificationQuestions": []
}
```

Excel result

The validated case becomes one row in the Test Cases worksheet. The sheet includes title and disclaimer rows, filters, frozen headers, wrapped text, priority styling, review status, and all twelve required test-case columns.

4. Validation Before Trust

- Reject blank, short, oversized, repeated-character, or unreadable input before an API call.
- Reject invalid JSON, missing fields, unknown requirement IDs, excessive case counts, and blocked cases.
- Reject vague expected results such as “behaves correctly” or “handled as expected”.
- Reject assumptions other than None so unsupported architecture or business behaviour cannot enter the workbook.
- Compare test-case meaning to detect substantial duplicates.
- Require traceability evidence to share meaningful terms with the supplied specification.
- Generate no workbook when any final case fails validation.

Human approval policy

Passing automated validation does not make AI output authoritative. A QA engineer must compare every case with the source specification, verify the business meaning and test data, review priority, confirm executability, and approve the workbook before execution.

Verified evidence

Lint, strict TypeScript checking, 19 automated tests, production dependency audit, desktop rendering, 390px mobile rendering, empty-input rejection, and a mocked successful browser download were verified. A live OpenRouter run then processed the supplied 774-character digital-wallet specification with a maximum of 20 cases. Nine cases passed validation and were exported to the sanitized example workbook under `ai/examples/`.

5. Commands and Limitations

```
cd ai
npm install
cp .env.example .env
npm start
```

Quality commands

```
npm run lint
npm run typecheck
npm test
npm audit --omit=dev
```

Limitations

- Only pasted text is accepted.
- The selected OpenRouter model must support strict JSON-schema output.
- OpenRouter availability, cost, rate limits, and provider behaviour remain external dependencies.
- Lexical traceability and duplicate checks reduce risk but cannot replace domain knowledge.
- Incomplete specifications may produce clarification questions and no downloadable workbook.

Final responsibility

AI proposes test cases; deterministic validation prevents known low-quality patterns; the QA engineer remains responsible for accepting, correcting, prioritising, and executing the final cases.